

02. Database - Học Và Nhớ Library74 Từ Gốc

Tài liệu này xem bạn là người mới học lập trình web ở mức cơ bản. Mục tiêu không phải học thuộc lòng, mà là hiểu database đang giải quyết vấn đề gì, vì sao thiết kế như vậy, và khi hội đồng hỏi thì bạn giải thích được bằng logic.

Lộ trình học toàn project:

1. FE - Frontend: người dùng nhìn thấy gì, bấm gì, dữ liệu đi đâu.
2. DB - Database: dữ liệu được lưu như thế nào.
3. BE - Backend: nghiệp vụ, bảo mật, API.
4. AI - Recommendation, semantic search, xử lý tri thức.
5. Deploy/DevOps - đưa hệ thống chạy thật trên server.

Trong file này ta học phần Database.

1. Database Là Gì?

1.1. Vấn Đề Thực Tế

Một hệ thống thư viện cần nhớ rất nhiều thứ:

- Ai là sinh viên, thủ thư, admin.
- Sách nào đang có trong thư viện.
- Mỗi sách có bao nhiêu bản sao.
- Bản sao nào đang được mượn.
- Ai đang mượn sách nào.
- Khi nào đến hạn trả.
- Ai bị phạt, phạt bao nhiêu.
- Người dùng đã wishlist, rating, tìm kiếm gì.
- Phiếu hỗ trợ đang ở trạng thái nào.
- Vector AI của sách nằm ở đâu.

Nếu chỉ lưu trong frontend hoặc biến tạm thời của backend, khi tắt app dữ liệu sẽ mất.

Database là nơi lưu dữ liệu lâu dài.

Nói đơn giản:

Frontend -> Backend -> Database

Frontend không chạm trực tiếp vào database. Backend là người kiểm soát việc đọc/ghi.

1.2. Định Nghĩa

Database là hệ thống lưu trữ dữ liệu có cấu trúc, cho phép thêm, đọc, sửa, xóa và truy vấn dữ liệu.

Trong Library74, database chính là:

PostgreSQL + pgvector

2. Vì Sao Dùng PostgreSQL?

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mạnh và phổ biến.

Project dùng PostgreSQL vì:

- Lưu dữ liệu quan hệ tốt: user, sách, bản sao, giao dịch.
- Hỗ trợ transaction để dữ liệu nhất quán.
- Hỗ trợ constraint như khóa chính, khóa ngoại, unique.
- Hỗ trợ index để truy vấn nhanh.
- Hỗ trợ JSONB cho dữ liệu linh hoạt.
- Có extension **pgvector** để lưu vector AI.

Trả lời hội đồng:

PostgreSQL phù hợp vì hệ thống thư viện có nhiều quan hệ dữ liệu chặt chẽ như user, sách, bản sao, giao dịch mượn và phí phạt. Ngoài ra PostgreSQL hỗ trợ pgvector, giúp lưu embedding AI ngay trong cùng hệ thống database.

3. Table, Row, Column Là Gì?

3.1. Table

Table là bảng lưu một loại dữ liệu.

Ví dụ:

```
users
publications
items
borrowing_transactions
fines
contact_messages
```

3.2. Row

Row là một dòng dữ liệu cụ thể.

Ví dụ một dòng trong bảng **users** là một người dùng.

3.3. Column

Column là một thuộc tính.

Ví dụ bảng **users** có các cột:

```
id
email
full_name
hashed_password
status
created_at
updated_at
```

Ví dụ dễ hiểu:

```
Table users
-> mỗi row là một tài khoản
-> mỗi column là một thông tin của tài khoản
```

4. Primary Key Là Gì?

4.1. Vấn Đề Thực Tế

Trong thư viện có thể có nhiều người tên giống nhau.

Ví dụ:

```
Nguyễn Văn A
Nguyễn Văn A
```

Nếu chỉ dựa vào tên, hệ thống không biết ai là ai.

4.2. Định Nghĩa

Primary Key, viết tắt là **PK**, là khóa chính dùng để định danh duy nhất một dòng trong bảng.

Ví dụ:

```
users.id
publications.id
items.id
borrowing_transactions.id
```

Khi nói `user_id = 15`, database biết chính xác đó là user nào.

Trả lời hội đồng:

Primary key giúp định danh duy nhất mỗi bản ghi. Trong hệ thống có thể có nhiều tên trùng nhau, nhưng id thì không trùng, nên backend dùng id để liên kết dữ liệu chính xác.

5. Foreign Key Là Gì?

5.1. Vấn Đề Thực Tế

Một giao dịch mượn sách cần biết:

- Ai mượn.
- Mượn bản sao nào.

Ta không nên copy toàn bộ thông tin user và item vào transaction. Ta chỉ cần lưu id tham chiếu.

5.2. Định Nghĩa

Foreign Key, viết tắt là **FK**, là khóa ngoại dùng để liên kết một bảng với bảng khác.

Ví dụ:

```
borrowing_transactions.user_id -> users.id
borrowing_transactions.item_id -> items.id
items.publication_id -> publications.id
```

Ý nghĩa:

- Transaction biết user nào mượn.
- Transaction biết item nào được mượn.
- Item biết nó thuộc đầu sách nào.

Trả lời hội đồng:

Foreign key giúp database đảm bảo dữ liệu liên kết hợp lệ. Ví dụ một transaction không được tham chiếu tới user không tồn tại hoặc item không tồn tại.

6. Unique, NOT NULL, CHECK Là Gì?

Database không chỉ lưu dữ liệu, mà còn bảo vệ dữ liệu.

6.1. Unique

UNIQUE đảm bảo giá trị không bị trùng.

Ví dụ:

```
users.email  
items.barcode  
publications.isbn  
roles.role_name
```

Ý nghĩa:

- Một email không thể đăng ký hai tài khoản.
- Một barcode không thể đại diện hai bản sao.

6.2. NOT NULL

NOT NULL nghĩa là bắt buộc có dữ liệu.

Ví dụ:

```
users.email  
users.full_name  
items.barcode  
items.publication_id
```

6.3. CHECK

CHECK giới hạn giá trị hợp lệ.

Ví dụ status user chỉ nên là:

```
ACTIVE  
INACTIVE  
LOCKED  
BANNED
```

Item status chỉ nên là:

```
AVAILABLE  
BORROWED  
RESERVED  
IN_MAINTENANCE  
LOST
```

Trả lời hội đồng:

Constraint giúp database tự bảo vệ tính đúng đắn dữ liệu. Backend có validate, nhưng database vẫn cần ràng buộc cuối cùng để tránh dữ liệu sai lọt vào.

7. Quan Hệ 1-1, 1-N, N-N

7.1. Quan Hệ 1-N

Một user có nhiều giao dịch mượn.

users 1 -> N borrowing_transactions

Một publication có nhiều item.

publications 1 -> N items

7.2. Quan Hệ N-N

Một sách có nhiều tác giả, một tác giả có nhiều sách.

publications N -> N authors

Database không lưu N-N trực tiếp. Nó dùng bảng nối:

publication_authors

Tương tự:

publication_categories
publication_tags
user_roles
fine_payment_order_fines

7.3. Quan Hệ 0..1

Một user thường có tối đa một wishlist.

```
users 0..1 -> 0..1 wish_lists
```

Trả lời hội đồng:

Quan hệ giúp mô hình hóa thế giới thật. Ví dụ một đầu sách có nhiều bản sao là 1-N, còn sách và tác giả là N-N nên cần bảng nối để tránh lặp dữ liệu và quản lý quan hệ rõ ràng.

8. ERD Là Gì?

ERD là Entity Relationship Diagram, sơ đồ mô tả các bảng và quan hệ giữa bảng.

Trong project có tài liệu:

```
LMS_BE/ERD_SPECIFICATION.md
```

Khi nhìn ERD, bạn cần đọc được:

- Bảng nào là thực thể chính.
- Bảng nào là bảng nối.
- Cột nào là khóa chính.
- Cột nào là khóa ngoại.
- Quan hệ là 1-N hay N-N.

Trả lời hội đồng:

ERD giúp trình bày cấu trúc dữ liệu tổng thể. Thay vì chỉ xem code, hội đồng có thể nhìn ERD để hiểu user, sách, mượn trả, phí phạt và AI liên kết với nhau như thế nào.

9. Các Nhóm Bảng Chính Trong Library74

Database được chia theo nhóm nghiệp vụ:

Nhóm	Bảng tiêu biểu	Mục đích
Người dùng/phân quyền	<code>users</code> , <code>roles</code> , <code>user_roles</code>	Tài khoản và quyền
Catalog	<code>publications</code> , <code>items</code> , <code>authors</code> , <code>publishers</code> , <code>categories</code> , <code>tags</code>	Dữ liệu sách

Nhóm	Bảng tiêu biểu	Mục đích
Bảng nối sách	<code>publication_authors</code> , <code>publication_categories</code> , <code>publication_tags</code>	Liên kết metadata
Mượn trả	<code>borrowing_transactions</code> , <code>reservations</code> , <code>finances</code>	Giao dịch thư viện
Thanh toán	<code>fine_payment_orders</code> , <code>fine_payment_order_fines</code>	PayOS/fine payment
Review	<code>ratings</code> , <code>rating_replies</code> , <code>rating_helpful_votes</code>	Đánh giá sách
Recommendation	<code>wish_lists</code> , <code>wish_lists_item</code> , <code>search_history</code> , <code>user_interactions</code>	Hành vi người dùng
Notification	<code>notifications</code> , <code>user_notifications</code>	Thông báo trong app
Auth token	<code>refresh_tokens</code> , <code>password_reset_tokens</code>	Đăng nhập/reset
Contact	<code>contact_messages</code> , <code>contact_message_comments</code> , <code>contact_internal_notes</code>	Hỗ trợ người dùng
Audit/Policy	<code>audit_logs</code> , policy tables	Theo dõi và cấu hình
AI	<code>publication_vectors</code> , <code>publication_etl_runs</code> , AI fields trong <code>publications</code>	Search/recommendation

10. Nhóm User Và Role

10.1. Bảng `users`

`users` là bảng trung tâm cho tài khoản.

Nó lưu:

- Email.
- Mật khẩu đã hash.
- Họ tên.
- Số điện thoại.
- Avatar.
- Mã sinh viên.
- Trạng thái tài khoản.
- Điểm uy tín, điểm đóng góp.

10.2. Bảng `roles`

`roles` lưu các vai trò:

```
ADMIN  
LIBRARIAN  
STUDENT
```

10.3. Bảng `user_roles`

Một user có thể có nhiều role, một role thuộc nhiều user.

Vì vậy cần bảng nối:

```
user_roles(user_id, role_id)
```

Trả lời hội đồng:

Project tách user và role để phân quyền linh hoạt. User không lưu role bằng text đơn giản, mà liên kết qua `user_roles` để có thể mở rộng nếu một tài khoản cần nhiều vai trò.

11. Vì Sao `hashed_password` Có Thể Nullable?

Một số user đăng nhập bằng Google OAuth2.

Với OAuth:

- User không đặt password trong hệ thống.
- Backend nhận danh tính từ Google.
- Vì vậy `hashed_password` có thể null.

Trả lời hội đồng:

`hashed_password` nullable vì hệ thống hỗ trợ OAuth. Tài khoản đăng nhập bằng Google không cần lưu password nội bộ, nhưng vẫn có user record để phân quyền và liên kết dữ liệu.

12. Nhóm Catalog: Sách Và Bản Sao

12.1. `publications`

`publications` là đầu sách hoặc metadata sách.

Nó lưu:

- Title.

- ISBN.
- Mô tả.
- Năm xuất bản.
- Ngôn ngữ.
- Ảnh bìa.
- URL PDF.
- AI summary.
- AI target audience.

12.2. `items`

`items` là bản sao cụ thể có thể mượn.

Nó lưu:

- Barcode.
- Publication id.
- Status.
- Condition.
- Branch.
- Location.

12.3. Vì Sao Tách Publication Và Item?

Ví dụ thư viện có sách "Clean Code".

```
Publication: Clean Code
Item 1: barcode CC001, AVAILABLE
Item 2: barcode CC002, BORROWED
Item 3: barcode CC003, LOST
```

Nếu gộp chung, rất khó biết mỗi bản sao đang ở đâu và trạng thái gì.

Trả lời hội đồng:

Publication là đầu sách, item là từng bản sao. Người dùng tìm theo publication, nhưng nghiệp vụ mượn trả phải theo item cụ thể để biết bản sao nào đang available, borrowed hay lost.

13. Nhóm Author, Category, Tag

13.1. Author

Một sách có thể có nhiều tác giả.

Một tác giả có thể viết nhiều sách.

Vì vậy:

```
publications N-N authors  
-> publication_authors
```

13.2. Category

Một sách có thể thuộc nhiều danh mục.

Một danh mục có nhiều sách.

Vì vậy:

```
publications N-N categories  
-> publication_categories
```

Category còn có thể có danh mục cha:

```
categories.parent_category_id -> categories.id
```

Đây là quan hệ tự tham chiếu.

13.3. Tag

Tag có thể do thủ thư tạo hoặc AI gợi ý.

Quan hệ:

```
publications N-N tags  
-> publication_tags
```

Trả lời hội đồng:

Author, category và tag đều có thể liên kết nhiều-nhiều với sách, nên project dùng bảng nối. Cách này tránh lặp dữ liệu và giúp tìm kiếm/lọc sách linh hoạt.

14. Nhóm Translation/i18n

Project có English mode nên cần dữ liệu dịch cho metadata.

Các bảng:

```
publication_translations
category_translations
tag_translations
```

Ý nghĩa:

- Dữ liệu gốc vẫn nằm ở bảng chính.
- Bản dịch là lớp bổ sung.
- Một publication có thể có nhiều bản dịch theo `language_code`.

Ví dụ:

```
publication_id = 10
language_code = en
title = "Database System Concepts"
```

Trả lời hội đồng:

Translation table giúp hỗ trợ đa ngôn ngữ mà không làm rối bảng chính. Dữ liệu gốc vẫn giữ nguyên, còn bản dịch được lưu theo `language_code` để frontend chọn hiển thị tiếng Việt hoặc tiếng Anh.

15. Nhóm Mượn Trả: `borrowing_transactions`

15.1. Bảng này Lưu Gì?

`borrowing_transactions` lưu lịch sử mượn/trả.

Nó liên kết:

- User mượn.
- Item được mượn.
- Thủ thư xác nhận mượn.
- Thủ thư xác nhận trả.
- Ngày mượn.
- Hạn trả.
- Ngày trả.
- Trạng thái.

15.2. Vì Sao Transaction Liên Kết Với Item, Không Chỉ Publication?

Vì mượn là mượn một bản sao cụ thể.

Ví dụ:

User mượn Clean Code bản barcode CC002

Không chỉ là "mượn Clean Code" chung chung.

Trả lời hội đồng:

Transaction liên kết với item vì trạng thái mượn trả thuộc về bản sao cụ thể. Nếu chỉ lưu publication_id, hệ thống không biết bản sao nào đang được giữ bởi người dùng.

16. Nhóm Đặt Trước: **reservations**

16.1. Vấn Đề Thực Tế

Khi sách chưa có bản sao rảnh, người dùng vẫn muốn xếp hàng.

reservations lưu:

- User đặt.
- Publication được đặt.
- Item được gán nếu đã có bản sao sẵn.
- Vị trí hàng đợi.
- Hạn giữ sách.
- Trạng thái.

16.2. Vì Sao Reservation Liên Kết Với Publication Và Có **assigned_item_id**?

Khi đặt trước, có thể chưa biết bản sao nào sẽ trả về trước.

Vì vậy ban đầu đặt theo publication. Khi có item available, hệ thống gán vào **assigned_item_id**.

Trả lời hội đồng:

Reservation theo publication vì lúc đặt chưa chắc biết bản sao nào sẽ sẵn sàng. Khi có bản sao trả về, hệ thống mới gán assigned_item_id để người dùng đến nhận.

17. Nhóm Phí Phạt Và Thanh Toán

17.1. **fines**

fines lưu khoản phạt phát sinh từ một giao dịch mượn.

Ví dụ:

- Trả trễ.
- Mất sách.
- Hồng sách.

Quan hệ:

```
borrowing_transactions 1-N fines
```

17.2. `fine_payment_orders`

Bảng này lưu đơn thanh toán PayOS.

Một order có thể thanh toán nhiều fine.

Vì vậy cần bảng nối:

```
fine_payment_order_fines
```

17.3. Vì Sao Không Chỉ Lưu `fine_ids` Dạng Text?

Project có thể lưu `fine_ids` như snapshot để hiển thị, nhưng quan hệ chuẩn vẫn nên nằm ở bảng nối.

Trả lời hội đồng:

```
fine_payment_order_fines là bảng nối chuẩn hóa giữa payment order và fine.
Snapshot fine_ids có thể phục vụ hiển thị hoặc trace, nhưng FK trong bảng
nối mới giúp database đảm bảo quan hệ đúng.
```

18. Nhóm Rating Và Review

18.1. `ratings`

`ratings` là review sách, khác với system review.

Nó lưu:

- User đánh giá.
- Publication được đánh giá.
- Số sao.
- Comment.
- Transaction nếu review gắn với lần mượn thật.
- `verified_borrow`.

18.2. Vì Sao Rating Có `transaction_id`?

Một user có thể mượn cùng một sách nhiều lần.

Nếu review gắn với transaction, hệ thống biết review đó đến từ lần mượn nào.

Sau migration mới, unique theo `transaction_id` khi có giá trị, thay vì khóa cứng một user chỉ review một sách một lần.

Trả lời hội đồng:

Rating có thể gắn với transaction để chứng minh người dùng đã mượn thật. Điều này giúp review đáng tin hơn và cho phép user đánh giá theo từng lần mượn nếu nghiệp vụ cần.

19. Nhóm Wishlist, Search History, User Interaction

19.1. Wishlist

Wishlist gồm:

```
wish_lists  
wish_lists_item
```

Một wishlist có nhiều publication, một publication có thể nằm trong wishlist của nhiều user.

19.2. Search History

`search_history` lưu những gì user đã tìm.

Dùng để:

- Hiện thị lịch sử.
- Phân tích hành vi.
- Có thể hỗ trợ recommendation.

19.3. User Interaction

`user_interactions` lưu hành vi:

```
WATCH  
WISHLIST  
BORROWED
```

AI recommendation dùng các hành vi này với trọng số khác nhau.

Trả lời hội đồng:

Wishlist và search history phục vụ trải nghiệm người dùng, còn user_interactions là dữ liệu hành vi cho AI recommendation. Hành vi mượn có giá trị mạnh hơn chỉ xem, nên AI có thể dùng trọng số khác nhau.

20. Nhóm Notification

Notification có hai bảng:

```
notifications
user_notifications
```

Ý nghĩa:

- **notifications**: nội dung thông báo gốc.
- **user_notifications**: thông báo đó thuộc user nào, đã đọc chưa.

Vì một notification có thể gửi cho nhiều user, và một user có nhiều notification, nên đây là quan hệ N-N qua bảng nối.

Trả lời hội đồng:

Project tách notifications và user_notifications để một nội dung thông báo có thể gửi cho nhiều người, đồng thời mỗi người có trạng thái đã đọc riêng.

21. Nhóm Auth Token

21.1. refresh_tokens

Lưu refresh token để user không phải đăng nhập lại liên tục.

Thông tin quan trọng:

- Token.
- User.
- Device id.
- Expiry date.
- Revoked.

21.2. password_reset_tokens

Lưu token reset password.

Thông tin quan trọng:

- Token.
- User.
- Expires at.

Trả lời hội đồng:

Token được lưu để backend kiểm soát phiên đăng nhập và reset password. Refresh token có thể bị revoke, còn reset token có hạn sử dụng để giảm rủi ro bảo mật.

22. Nhóm Contact Ticket

22.1. Vấn Đề Thực Tế

Người dùng cần gửi yêu cầu hỗ trợ và theo dõi trạng thái.

Các bảng contact lưu:

```
contact_messages
contact_message_comments
contact_internal_notes
```

22.2. `contact_messages`

Lưu phiếu hỗ trợ chính:

- Ticket code.
- Người gửi.
- Category.
- Subject.
- Message.
- Status.
- Người xử lý.
- Feedback rating.
- Reply.

22.3. Comment Và Internal Note Khác Nhau Thế Nào?

Comment là trao đổi với user.

Internal note là ghi chú nội bộ của thủ thư/admin, user không thấy.

Trả lời hội đồng:

Contact ticket giúp biến form liên hệ thành quy trình hỗ trợ có trạng thái. Comment là trao đổi công khai với người dùng, còn internal note là ghi chú

nội bộ để thủ thư phối hợp xử lý.

23. Nhóm Audit Log Và Policy

23.1. Audit Log

Audit log ghi lại hành động quan trọng:

- Ai thao tác.
- Thao tác gì.
- Trên đối tượng nào.
- Thời điểm nào.

Mục tiêu:

- Truy vết.
- Kiểm tra sai sót.
- Tăng tính minh bạch khi admin/thủ thư thao tác.

23.2. Policy

Policy là cấu hình nghiệp vụ, ví dụ:

- Số sách tối đa được mượn.
- Số ngày mượn.
- Mức phạt.
- Quy định theo vai trò/campus nếu có.

Trả lời hội đồng:

Audit log giúp hệ thống có khả năng truy vết hành động quản trị. Policy giúp một số quy định nghiệp vụ có thể cấu hình bằng dữ liệu thay vì hardcoded toàn bộ trong code.

24. Nhóm AI Trong Database

Database còn lưu dữ liệu phục vụ AI:

```
publication_vectors
publication_etl_runs
publications.ai_summary
publications.ai_target_audience
publication_tags
user_interactions
```

24.1. Vector

`publication_vectors` lưu:

- Publication id.
- Chunk text.
- Embedding vector.

Vector dùng cho semantic search.

24.2. ETL Runs

`publication_etl_runs` lưu hash file PDF.

Nếu PDF không đổi, AI có thể skip xử lý lại.

24.3. AI Fields Trong Publications

`publications` có:

```
ai_summary  
ai_target_audience
```

Đây là kết quả AI để frontend hiển thị.

Trả lời hội đồng:

Database không chỉ lưu dữ liệu nghiệp vụ truyền thống mà còn lưu vector và kết quả AI. Vector phục vụ máy tìm kiếm theo ngữ nghĩa, còn ai_summary và tags phục vụ người dùng đọc hiểu nhanh.

25. Index Là Gì?

25.1. Vấn Đề Thực Tế

Nếu bảng có nhiều dữ liệu, tìm kiếm bằng cách quét toàn bộ bảng sẽ chậm.

Index giống mục lục sách.

Thay vì đọc từng trang, ta tra mục lục để đến đúng trang.

25.2. Index Trong Project

Project có index cho:

- Email user.
- Title publication.
- Description/AI summary.

- Barcode item.
- Computed time của recommendation.
- Vector search bằng HNSW/pgvector.

25.3. Có Phải Càng Nhiều Index Càng Tốt?

Không.

Index giúp đọc nhanh hơn nhưng làm ghi chậm hơn và tốn dung lượng.

Trả lời hội đồng:

Index giúp tăng tốc truy vấn thường dùng như tìm user theo email, tìm sách theo title hoặc tìm vector gần nhất. Nhưng không nên tạo quá nhiều index vì mỗi lần insert/update database phải cập nhật index, làm ghi dữ liệu chậm hơn.

26. Normalization Là Gì?

26.1. Vấn Đề Thực Tế

Nếu lưu tác giả trực tiếp trong từng sách:

```
Clean Code – Robert C. Martin  
Agile Software Development – Robert C. Martin
```

Tên tác giả bị lặp nhiều lần. Nếu sửa sai chính tả, phải sửa nhiều dòng.

26.2. Định Nghĩa

Normalization là thiết kế database để giảm lặp dữ liệu và giữ quan hệ rõ ràng.

Ví dụ:

```
authors  
publications  
publication_authors
```

Tên tác giả chỉ lưu một lần trong **authors**.

26.3. Có Khi Nào Lưu Snapshot Không?

Có.

Ví dụ payment order có thể lưu snapshot:

```
student_id
fine_ids
description
```

Vì thông tin thanh toán cần giữ dấu vết tại thời điểm tạo order.

Trả lời hội đồng:

Project chủ yếu chuẩn hóa để tránh lặp dữ liệu, ví dụ sách và tác giả tách bảng nối. Nhưng một số nghiệp vụ như payment order vẫn lưu snapshot để giữ lịch sử giao dịch tại thời điểm thanh toán.

27. Transaction Database Là Gì?

27.1. Vấn Đề Thực Tế

Khi trả sách, hệ thống có thể phải:

- Cập nhật transaction.
- Cập nhật item.
- Tạo fine nếu trễ.
- Tạo notification.

Nếu làm được bước 1 nhưng lỗi ở bước 2, dữ liệu sai.

27.2. ACID

Transaction database giúp đảm bảo ACID:

Chữ	Ý nghĩa đơn giản
A - Atomicity	Thành công hết hoặc rollback hết
C - Consistency	Dữ liệu vẫn hợp lệ
I - Isolation	Các giao dịch không giẫm lên nhau
D - Durability	Commit rồi thì dữ liệu được lưu bền vững

Trả lời hội đồng:

Database transaction rất quan trọng với mượn trả vì một nghiệp vụ ảnh hưởng nhiều bảng. Nếu có lỗi giữa chừng, rollback giúp tránh trạng thái dữ liệu nửa đúng nửa sai.

28. Flyway Migration Là Gì?

28.1. Vấn Đề Thực Tế

Database thay đổi liên tục:

- Thêm bảng.
- Thêm cột.
- Thêm index.
- Sửa constraint.
- Seed dữ liệu.

Nếu sửa tay trên từng máy, database local và production dễ lệch nhau.

28.2. Flyway Trong Project

Migration nằm ở:

```
LMS_BE/library-bootstrap/src/main/resources/db/migration/
```

Ví dụ:

```
V1__create_schema.sql
V15__fine_payment_orders.sql
V20__schema_cleanup_and_ai_indexes.sql
V34__enable_unaccent_for_search.sql
V35__contact_messages.sql
V39__contact_ticket_notifications.sql
V42__demo_slide_account_roles.sql
```

Khi backend start:

```
Flyway kiểm tra migration nào chưa chạy
-> chạy theo thứ tự version
-> ghi lịch sử vào flyway_schema_history
```

Trả lời hội đồng:

Flyway giúp quản lý schema database bằng file migration có version. Nhờ vậy mọi môi trường chạy cùng lịch sử thay đổi database và tránh sửa tay khó kiểm soát.

29. Vì Sao `flyway_schema_history` Không Vẽ Vào ERD Nghiệp Vụ?

`flyway_schema_history` là bảng kỹ thuật.

Nó lưu:

- Migration nào đã chạy.
- Chạy lúc nào.
- Thành công hay không.

Nó không phải nghiệp vụ thư viện.

Trả lời hội đồng:

`flyway_schema_history` là bảng kỹ thuật để Flyway quản lý migration, không phải thực thể nghiệp vụ. Vì vậy khi vẽ ERD nghiệp vụ, em không đưa bảng này vào nhóm chính.

30. JPA Entity Và Database Table Liên Quan Thế Nào?

Backend Java có Entity:

```
UserEntity  
PublicationEntity  
ItemEntity  
BorrowingTransactionEntity  
FineEntity  
ReservationEntity  
RatingEntity
```

Entity là class Java đại diện cho bảng database.

Ví dụ:

```
UserEntity -> users  
PublicationEntity -> publications  
ItemEntity -> items
```

Backend dùng Entity để làm việc với dữ liệu bằng object, còn database lưu bằng bảng.

Trả lời hội đồng:

Entity là cách backend ánh xạ bảng database sang object Java. Database vẫn là nơi lưu thật, còn Entity giúp code Java thao tác dữ liệu dễ hơn.

31. Dữ Liệu Đi Từ Frontend Xuống Database Như Thế Nào?

Ví dụ user tạo phiếu hỗ trợ:

1. Frontend gửi POST /api/v1/contact-messages
2. Backend kiểm tra token
3. Backend validate subject/message
4. Backend tạo ticket_code
5. Backend insert vào contact_messages
6. Backend insert notification nếu cần
7. Database lưu dữ liệu
8. Backend trả response
9. Frontend hiển thị phiếu mới

Ví dụ mượn sách:

1. Frontend gửi yêu cầu mượn publication
2. Backend tìm item AVAILABLE
3. Backend tạo borrowing_transactions
4. Backend đổi item.status
5. Database commit transaction
6. Backend trả kết quả

Trả lời hội đồng:

Frontend không ghi database trực tiếp. Backend nhận request, kiểm tra quyền và nghiệp vụ, sau đó mới đọc/ghi database trong transaction nếu cần.

32. Xóa Dữ Liệu Và ON DELETE

32.1. Vấn Đề Thực Tế

Nếu xóa một publication, các bảng liên quan xử lý thế nào?

Ví dụ:

- Bản dịch của publication.
- Tag liên kết.
- Review.
- Item.
- Transaction lịch sử.

Không thể xóa bừa vì có thể mất lịch sử mượn trả.

32.2. CASCADE, RESTRICT, SET NULL

Các kiểu thường gặp:

Kiểu	Ý nghĩa
CASCADE	Xóa cha thì xóa con
RESTRICT	Không cho xóa nếu còn con
SET NULL	Xóa cha thì FK con thành null

Ví dụ:

- Translation có thể **ON DELETE CASCADE** theo publication.
- Fine bị payment order tham chiếu có thể cần restrict.
- Một số librarian reference có thể nullable.

Trả lời hội đồng:

Không phải quan hệ nào cũng cascade. Với dữ liệu lịch sử như mượn trả và thanh toán, cần giữ lại để truy vết. Cascade phù hợp hơn với dữ liệu phụ thuộc như bản dịch hoặc bảng nối.

33. Backup Và Production Database

Database production là phần quan trọng nhất vì chứa dữ liệu thật.

Nguyên tắc:

- Không chạy **docker compose down -v** trên production.
- Không chạy **flyway clean** khi có dữ liệu thật.
- Backup trước khi migration rủi ro.
- Không commit **.env.prod**.
- Không sửa DB production bằng tay nếu chưa biết tác động.

Backup trong project có thể chạy qua compose profile:

```
docker compose --env-file .env.prod -f docker-compose.prod.yml --profile backup run --rm postgres-backup
```

Trả lời hội đồng:

Database production cần được bảo vệ nghiêm ngặt vì mất database là mất dữ liệu nghiệp vụ. Project tránh xóa volume, dùng Flyway migration có kiểm soát và nên backup trước thay đổi rủi ro.

34. Cách Tự Kiểm Tra Database

34.1. Kết Nối Local

Backend local thường dùng:

```
psql -h localhost -p 5432 -U library -d library
```

34.2. Một Số Query Dễ Hiểu

Đếm user:

```
SELECT COUNT(*) FROM users;
```

Đếm sách:

```
SELECT COUNT(*) FROM publications;
```

Xem một sách có bao nhiêu bản sao:

```
SELECT p.id, p.title, COUNT(i.id) AS item_count
FROM publications p
LEFT JOIN items i ON i.publication_id = p.id
GROUP BY p.id, p.title
ORDER BY item_count DESC
LIMIT 10;
```

Xem transaction đang mượn:

```
SELECT bt.id, u.email, p.title, i.barcode, bt.status, bt.due_date
FROM borrowing_transactions bt
JOIN users u ON u.id = bt.user_id
JOIN items i ON i.id = bt.item_id
JOIN publications p ON p.id = i.publication_id
WHERE bt.status IN ('WAITING_FOR_PICKUP', 'BORROWING', 'OVERDUE')
ORDER BY bt.due_date ASC;
```

Xem phiếu hỗ trợ mới:

```
SELECT id, ticket_code, subject, status, created_at
FROM contact_messages
```

```
ORDER BY created_at DESC  
LIMIT 10;
```

35. Những Câu Hỏi Đồng Hay Hỏi

Câu 1: Vì sao cần database?

Trả lời:

Database lưu dữ liệu lâu dài và có cấu trúc. Hệ thống thư viện cần lưu user, sách, bản sao, giao dịch mượn trả, phí phạt, review, notification và dữ liệu AI. Nếu không có database, dữ liệu sẽ mất khi app dừng.

Câu 2: Vì sao dùng database quan hệ?

Trả lời:

Vì dữ liệu thư viện có nhiều quan hệ rõ ràng: user mượn item, item thuộc publication, publication có author/category/tag, fine thuộc transaction. Database quan hệ giúp mô hình hóa và ràng buộc các quan hệ này tốt.

Câu 3: Primary key và foreign key khác nhau thế nào?

Trả lời:

Primary key định danh duy nhất một dòng trong bảng. Foreign key là khóa tham chiếu sang bảng khác để tạo quan hệ và đảm bảo dữ liệu liên kết hợp lệ.

Câu 4: Vì sao cần bảng nối?

Trả lời:

Bảng nối dùng cho quan hệ nhiều-nhiều. Ví dụ một sách có nhiều tác giả và một tác giả có nhiều sách, nên cần publication_authors để liên kết hai bảng mà không lặp dữ liệu.

Câu 5: Vì sao tách **publications** và **items**?

Trả lời:

Publication là đầu sách, item là bản sao cụ thể. Một đầu sách có nhiều bản sao, mỗi bản sao có barcode và status riêng. Tách như vậy giúp quản lý mượn trả chính xác.

Câu 6: Vì sao cần Flyway?

Trả lời:

Flyway quản lý thay đổi schema bằng migration version. Nhờ vậy local, test và production có cùng lịch sử database, tránh sửa tay gây lệch môi trường.

Câu 7: Vì sao không lưu tất cả vào một bảng lớn?

Trả lời:

Một bảng lớn sẽ lạp dữ liệu, khó cập nhật và khó đảm bảo quan hệ. Tách bảng theo thực thể như users, publications, items, transactions giúp dữ liệu rõ ràng, ít trùng lạp và dễ truy vấn.

Câu 8: Index dùng để làm gì?

Trả lời:

Index giúp truy vấn nhanh hơn, giống mục lục sách. Project dùng index cho các cột hay tìm kiếm như email, title, barcode và vector. Nhưng không tạo quá nhiều index vì nó làm insert/update chậm hơn.

Câu 9: Database hỗ trợ AI như thế nào?

Trả lời:

Database lưu vector embedding bằng pgvector, lưu summary/tags do AI tạo và lưu hành vi user cho recommendation. Nhờ vậy AI service có dữ liệu để semantic search và gợi ý sách.

Câu 10: Vì sao production DB cần backup?

Trả lời:

Production DB chứa dữ liệu thật. Nếu migration lỗi hoặc thao tác nhầm, backup giúp khôi phục. Vì vậy không chạy lệnh xóa volume và cần backup

trước thay đổi rủi ro.

36. Cách Tự Học Database Trong Project

Không nên học thuộc từng cột. Học theo nhóm nghiệp vụ.

36.1. Nhóm User/Auth

Đọc:

```
users
roles
user_roles
refresh_tokens
password_reset_tokens
```

Câu hỏi cần trả lời:

- User có những role nào?
- Refresh token dùng để làm gì?
- Vì sao hashed_password nullable?

36.2. Nhóm Catalog

Đọc:

```
publications
items
authors
categories
tags
publication_authors
publication_categories
publication_tags
```

Câu hỏi cần trả lời:

- Publication khác item thế nào?
- Vì sao author/category/tag dùng bảng nối?
- ISBN và barcode khác nhau thế nào?

36.3. Nhóm Circulation

Đọc:

```
borrowing_transactions  
reservations  
fines  
fine_payment_orders  
fine_payment_order_fines  
transaction_notes
```

Câu hỏi cần trả lời:

- Khi mượn sách thì bảng nào thay đổi?
- Fine phát sinh từ đâu?
- Một payment order liên kết nhiều fine thế nào?

36.4. Nhóm AI/Recommendation

Đọc:

```
user_interactions  
search_history  
wish_lists  
wish_lists_item  
publication_vectors  
publication_etl_runs
```

Câu hỏi cần trả lời:

- AI dùng hành vi nào?
- Vector lưu để làm gì?
- Hash PDF giúp gì?

36.5. Nhóm Contact/Notification

Đọc:

```
contact_messages  
contact_message_comments  
contact_internal_notes  
notifications  
user_notifications
```

Câu hỏi cần trả lời:

- Phiếu hỗ trợ có trạng thái gì?
- Comment khác internal note thế nào?
- Vì sao notification cần bảng nối user_notifications?

37. Bản Tóm Tắt Một Phút

Nếu phải giải thích database Library74 trong một phút:

Database Library74 dùng PostgreSQL để lưu dữ liệu nghiệp vụ thư viện và pgvector để hỗ trợ AI. Schema được chia theo nhóm: user/role cho tài khoản và phân quyền, catalog cho sách và bản sao, circulation cho mượn trả/đặt trước/phí phạt, recommendation cho wishlist/search/history/user interactions, contact cho phiếu hỗ trợ, notification cho thông báo và AI cho vector/summary/tag.

Thiết kế dùng primary key để định danh bản ghi, foreign key để liên kết bảng, unique/check/not null để bảo vệ dữ liệu. Publication được tách khỏi item vì một đầu sách có nhiều bản sao cụ thể. Các quan hệ nhiều-nhiều như sách-tác giả, sách-tag, user-role dùng bảng nối.

Database được quản lý bằng Flyway migration để mọi môi trường có cùng lịch sử schema. Production cần backup và không được xóa volume tùy tiện. Backend là lớp duy nhất đọc/ghi database sau khi kiểm tra quyền và nghiệp vụ.

38. Điều Quan Trọng Cần Nhớ

Nếu chỉ nhớ 10 ý, hãy nhớ:

1. Database lưu dữ liệu lâu dài của hệ thống.
2. PostgreSQL phù hợp vì Library74 có nhiều quan hệ dữ liệu.
3. Primary key định danh duy nhất một dòng.
4. Foreign key liên kết bảng và bảo vệ quan hệ.
5. Publication là đầu sách, Item là bản sao cụ thể.
6. Bảng nối dùng cho quan hệ nhiều-nhiều.
7. Constraint giúp dữ liệu không sai.
8. Index giúp truy vấn nhanh nhưng không nên lạm dụng.
9. Flyway quản lý thay đổi schema bằng migration version.
10. Production DB phải backup và không được xóa volume tùy tiện.